

**Mohsin Iban Hossain**

**AIUB, Compiler Final-Term Notes**

# COMPILER DESIGN

# Table of Contents

[illegible]

# REGULAR EXPRESSION

# Regular Expression

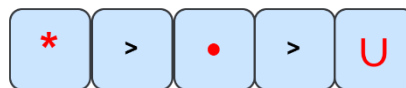
>>Regular expressions are used to represent certain sets of strings in an **algebraic** fashion, moving beyond simple English representations of strings.

- Structure:

Built using alphabets and **regular operations**:

- \* (star/**Closure**) – Represents zero or more occurrences of a string.
- • (Concatenation) – joining two patterns
- U (Union) – either/or (can also be written as )

- Precedence Order:



>>Examples:

1.  $(0\cup 1)0^*$

- Means: Starts with 0 or 1, followed by any number of 0s
- Language: {w | w starts with 0 or 1, followed by 0s}

2.  $(0\cup 1)^*$

- Means: Any combination of 0s and 1s
- Language: {all binary strings}

3.  $(0\cup 1)1$

- **Means:** Either 0 or 1, followed by 1
- Language: {w | w ends with 1, and starts with 0 or 1}

# Designing Regular Expressions

 **Example1:** Write the Regular Expression of the following languages, where  $\Sigma = \{a, b\}$ ,  
 “Language Accepting **Strings of Length Exactly Two**”

| Step 1: Understand the Alphabet                                                                                                                                                     | Step 2: Understand the Language Requirement                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><u>We are working with:</u></p> <p><math>\Sigma = \{a, b\}</math></p> <p>This means only the characters <b>a</b> and <b>b</b> can appear in our strings.</p>                     | <p>»The language accepts only strings of <b>length exactly two</b>.</p> <p>This means strings like:</p> <ul style="list-style-type: none"> <li>• aa</li> <li>• ab</li> <li>• ba</li> <li>• bb</li> </ul> <p>Only these four strings are valid.<br/>             Any string with <b>length <math>\neq 2</math></b> is not allowed.</p> |
| Step 3: List All Valid Strings                                                                                                                                                      | Step 4: Write the Regular Expression                                                                                                                                                                                                                                                                                                  |
| <p>»As we saw above, valid strings of length 2 over <math>\{a, b\}</math> are:</p> <ul style="list-style-type: none"> <li>• aa</li> <li>• ab</li> <li>• ba</li> <li>• bb</li> </ul> | <p><math>\therefore R = (aa \cup ab \cup ba \cup bb)</math> Or</p> <p><math>\therefore R = (aa \mid ab \mid ba \mid bb)</math></p>                                                                                                                                                                                                    |

 **Example2:** Write the Regular Expression of the following languages, where  $\Sigma = \{a, b\}$ ,  
 “which accepts **all strings of length at least two**.”

| Step 1: Understand the Alphabet                                                                                                                                                                                                                            | Step 2: Understand the Language Requirement                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><u>We are working with:</u></p> <p><math>\Sigma = \{a, b\}</math></p> <p>This means only the characters <b>a</b> and <b>b</b> can appear in our strings.</p>                                                                                            | <p>»The language accepts <b>all strings of length at least two</b></p> <p>This means:</p> <ul style="list-style-type: none"> <li>• The string must be <b>two or more characters long</b>.</li> <li>• This includes all strings of length 2, 3, 4, and so on.</li> </ul> |
| Step 3: List All Valid Strings                                                                                                                                                                                                                             | Step 4: Write the Regular Expression                                                                                                                                                                                                                                    |
| <p>» For strings of length at least two:</p> <ul style="list-style-type: none"> <li>• <b>Valid examples:</b> aa, ab, ba, bb, aaa, aab, abb, baa, bab, bba, bbb, etc.</li> <li>• <b>Invalid examples</b> (strings of length less than two): a, b</li> </ul> | <p><math>\therefore R = (a \mid b)(a \mid b)(a \mid b)^*</math></p>                                                                                                                                                                                                     |

### Explanation:

- $(a \mid b)$ : The first character can be either  $a$  or  $b$ .
- $(a \mid b)$ : The second character can also be either  $a$  or  $b$ .
- $(a \mid b)^*$ : After the second character, any number (including zero) of  $a$  or  $b$  characters can follow. This makes the string longer than two characters if needed.



**Example3:** Write the Regular Expression of the following languages, where  $\Sigma = \{a, b\}$ ,  
“which accepts **Strings of Length At Most Two.**”

| Step 1: Understand the Alphabet                                                                                                                                                                                                                                                             | Step 2: Understand the Language Requirement                                                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><u>We are working with:</u></p> $\Sigma = \{a, b\}$ <p>This means only the characters <math>a</math> and <math>b</math> can appear in our strings.</p>                                                                                                                                   | <p>&gt;&gt;The language accepts <b>all strings of length at Most two.</b></p> <p><b>This means:</b></p> <ul style="list-style-type: none"><li>• Length 0 (the <b>empty string</b>, denoted by <math>\epsilon</math>)</li><li>• Length 1 (like <math>a, b</math>)</li><li>• Length 2 (like <math>aa, ab, ba, bb</math>)</li></ul> |
| Step 3: List All Valid Strings                                                                                                                                                                                                                                                              | Step 4: Write the Regular Expression                                                                                                                                                                                                                                                                                             |
| <p>&gt;&gt; For strings of length at <b>Most</b> two:</p> <ul style="list-style-type: none"><li>• <math>\epsilon</math></li><li>• <math>a</math></li><li>• <math>b</math></li><li>• <math>aa</math></li><li>• <math>ab</math></li><li>• <math>ba</math></li><li>• <math>bb</math></li></ul> | $\therefore R = \epsilon \mid a \mid b \mid (a \mid b)(a \mid b)$                                                                                                                                                                                                                                                                |

## Practice Problem

 **Problem1:** Write the Regular Expression of the following languages, where  $\Sigma = \{a,b\}$ ,  
“Language Accepting **Strings of Length Exactly Two**”

→ **Here,**

»  $\Sigma = \{a,b\}$ ,

»  $L(M) = \{aa, ab, ba, bb\}$

$$\therefore R = (a \cup b) (a \cup b)$$

Or

$$\therefore R = (a \mid b) (a \mid b)$$

 **Problem2:** Write the Regular Expression of the following languages, where  $\Sigma = \{a,b\}$ ,  
“which accepts **all strings of length at least two.**”

→ **Here,**

»  $\Sigma = \{a,b\}$ ,

»  $L(M) = \{aa, ab, ba, bb, aaa, aab, abab..., bbaa..., \}$

$$\therefore R = (a \mid b) (a \mid b) (a \mid b)^*$$

 **Problem3:** Write the Regular Expression of the following languages, where  $\Sigma = \{a,b\}$ ,  
“which accepts **all strings of length at Most two.**”

→ **Here,**

»  $\Sigma = \{a,b\}$ ,

»  $L(M) = \{\epsilon, a, b, aa, bb, ab, ba\}$

$$\therefore R = \epsilon \mid a \mid b \mid (a \mid b) (a \mid b)$$

 **Problem4:** Write the Regular Expression of the following languages, where  $\Sigma = \{a,b\}$ ,  
“ $L = \{w \mid \text{each } a \text{ in } w \text{ is followed by at least two } b\}$ ”

→ **Here,**

»  $\Sigma = \{a,b\}$ ,

»  $L(M) = \{abb, b...abb, abbb..., abbabb..., abbbabb...\}$

$$\therefore R = b^* abb b^* (abb b^*)^*$$

Let's check whether the string “**abbabb**” matches the regular expression for the language:

1. **Breakdown of abbabb:**

- The string can be grouped as: a b b a b b  
(Each a is followed by exactly two bs, which satisfies the condition.)

2. **Matching with  $b^* a b b b^* (a b b b^*)^*$ :**

- $b^* \rightarrow$  Matches 0 leading bs (empty).
- $a b b b^* \rightarrow$  Matches a b b (the first  $b^*$  matches 0 trailing bs here).
- $(a b b b^*)^* \rightarrow$  Matches the remaining a b b (again,  $b^*$  matches 0 trailing bs).
- **Result:** The entire string abbabb is matched.

 **Problem5:** Write the Regular Expression of the following languages, where  $\Sigma = \{a,b\}$ ,  
“ $L = \{w \mid w \text{ contains odd number of } b\}$ ”

→ **Here,**

»  $\Sigma = \{a,b\}$ ,

»  $L(M) = \{b, a...b, ba..., bababa..., ba...ba...ba...\}$

$$\therefore R = a^* b (a^* b a^* b a^*)^*$$

 **Problem5:** Write the Regular Expression of the following languages, where  $\Sigma = \{a,b\}$ ,  
 “ $L = \{w \mid w \text{ contains } \textbf{abbab} \text{ substring}\}$ ”

➔ **Here,**

»  $\Sigma = \{a,b\}$ ,

»  $L(M) = \{\text{abbab}, a..abbab, b..abbab, \text{abbaba}..., \text{abbabb}...\}$

$$\therefore R = (a \cup b)^* \text{abbab} (a \cup b)^*$$

 **Problem6:** Write the Regular Expression of the following languages, where  $\Sigma = \{m,n\}$

- i. “ $\{w \mid \text{each } m \text{ in } w \text{ is followed by at least three } n\}$ ”
- ii. “ $\{w \mid w \text{ has odd length and starts with } m\}$ ”
- iii. “ $\{w \mid w \text{ contains 'mnnmmn' substring}\}$ ”

(i)

➔ **Here,**

»  $\Sigma = \{m, n\}$ ,

»  $L(M) = \{mnnnn, n...mnnnn, mnnnn..., mnnmmnn... \}$

$$\therefore R = n^* mnnnn n^* (mnnnn n^*)^*$$

(ii)

➔ **Here,**

»  $\Sigma = \{m, n\}$ ,

»  $L(M) = \{m, mmn, mnn, mnnnnm, ... \}$

$$\therefore R = m((m \cup n)(m \cup n))^*$$

(iii)

→ Here,

»  $\Sigma = \{m, n\}$ ,

»  $L(M) = \{ mnnmmn, mmn...mnnmmn, mnnmmn...mnnnm, ... \}$

$$\therefore R = (m|n)^* mnnmmn (m|n)^*$$

**Problem7:** Give the **Description** of the following Regular Expressions.

Consider  $\Sigma = \{x, y\}$

- i.  $(xUy)^* xyyxxy$
- ii.  $x^* y x^* y x^* y y^*$
- iii.  $(x \cup y). ((x \cup y). (x \cup y))^*$

(i)

→  $(xUy)^* xyyxxy$ :

- Language:  $\{w \mid w \text{ is a string over } \Sigma = \{x, y\} \text{ that ends with "xyyxy"}\}$ .

(ii)

→  $x^* y x^* y x^* y y^*$ :

- Language:  $\{w \mid w \text{ is a string over } \Sigma = \{x, y\} \text{ with at least three 'y's, and the third 'y' is followed by zero or more 'y's}\}$ .

(iii)

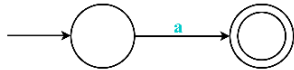
→  $(x \cup y). ((x \cup y). (x \cup y))^*$

- Language:  $\{w \mid w \text{ string starts with } x \text{ or } y, \text{ string has odd number of length}\}$

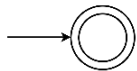
# CONVERSION OF RE TO NFA

# Formal procedure of Regular Language

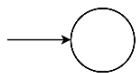
1.  $R = a$  ;  $a \in \Sigma$



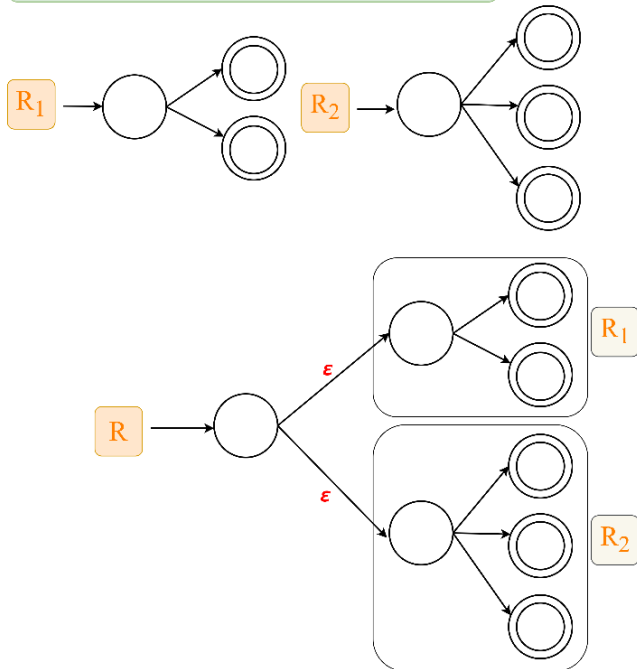
2.  $R = \epsilon$  ; initial state is the final state



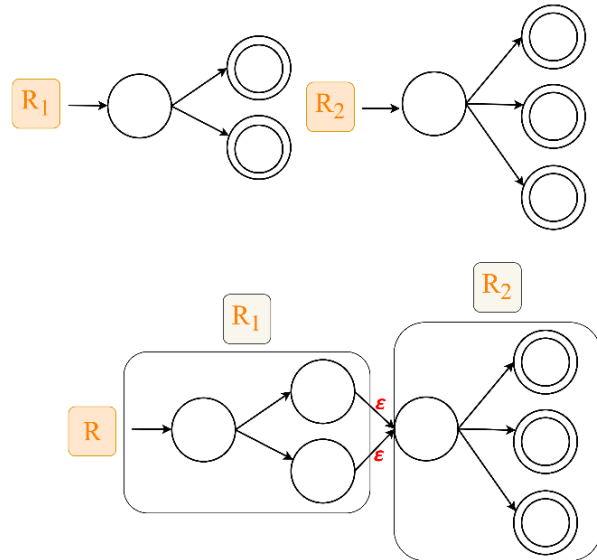
3.  $R = \phi$



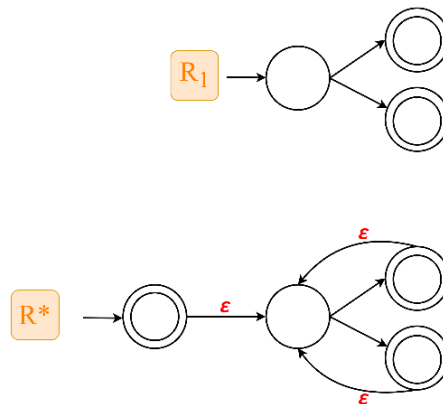
4.  $R = R_1 \cup R_2$  or  $R = R_1 | R_2$  or  $R = R_1$  or  $R_2$



5.  $R = R_1 \circ R_2$  or  $R = R_1 R_2$  or  $R = R_1$  Followed by  $R_2$

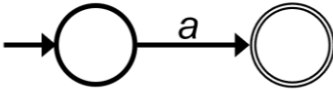
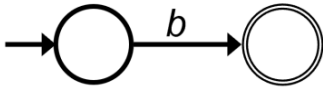
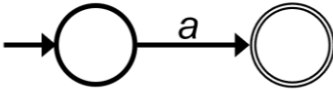
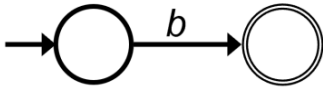
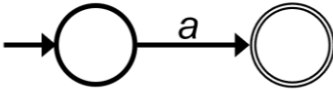
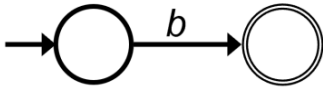
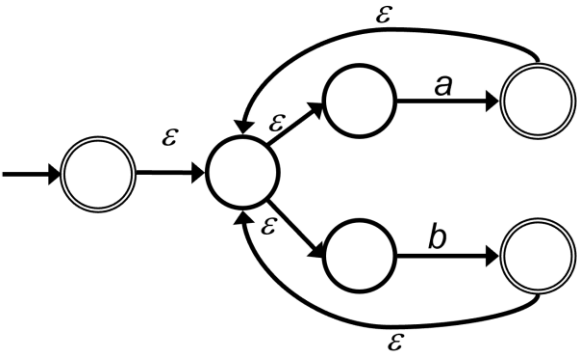
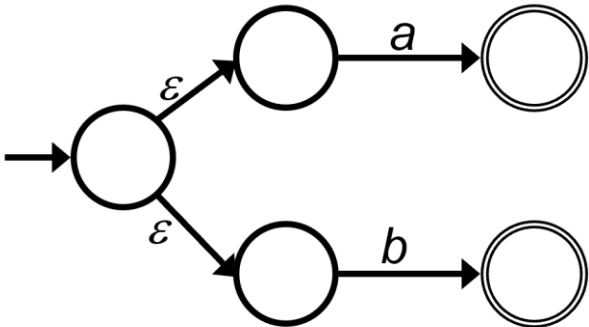
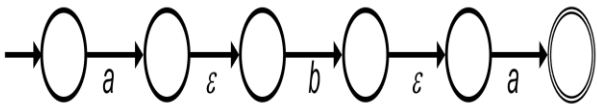
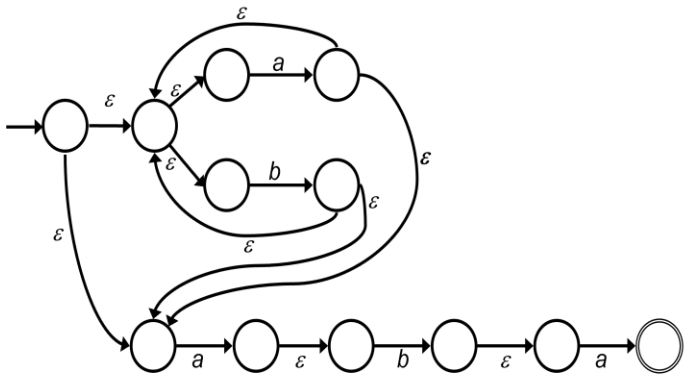


6.  $R^* = R_1^*$



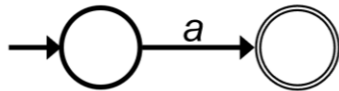
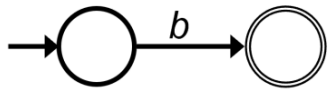
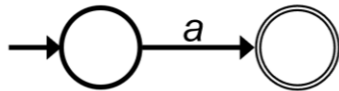
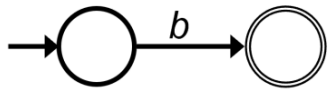
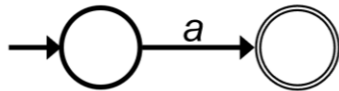
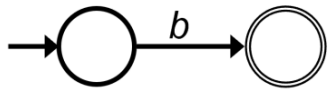
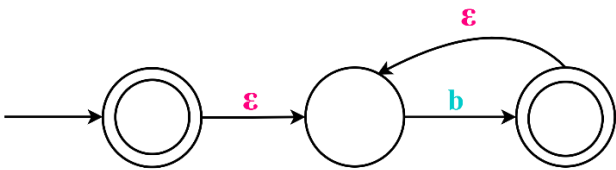
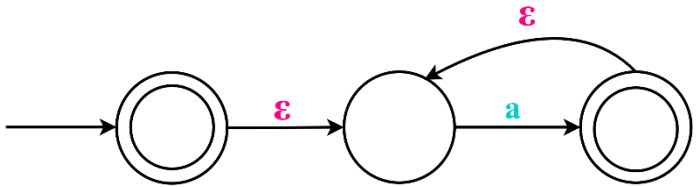
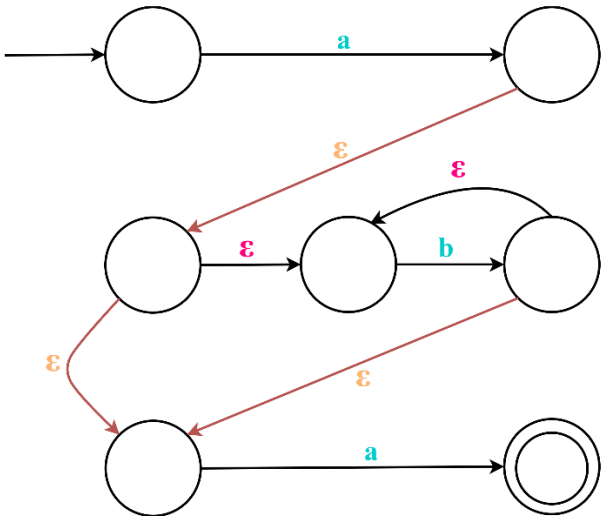
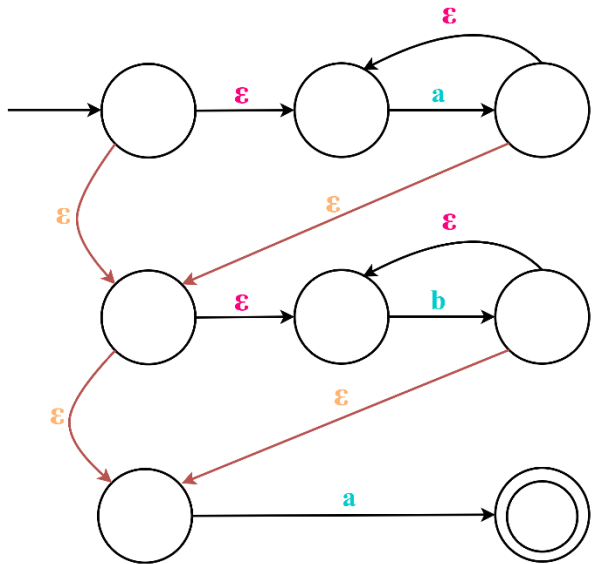
## Practice Problem

 **Problem1:** Convert the following Regular Expression into equivalent NFA using formal procedure. “ $(a \cup b)^* aba$ ”.

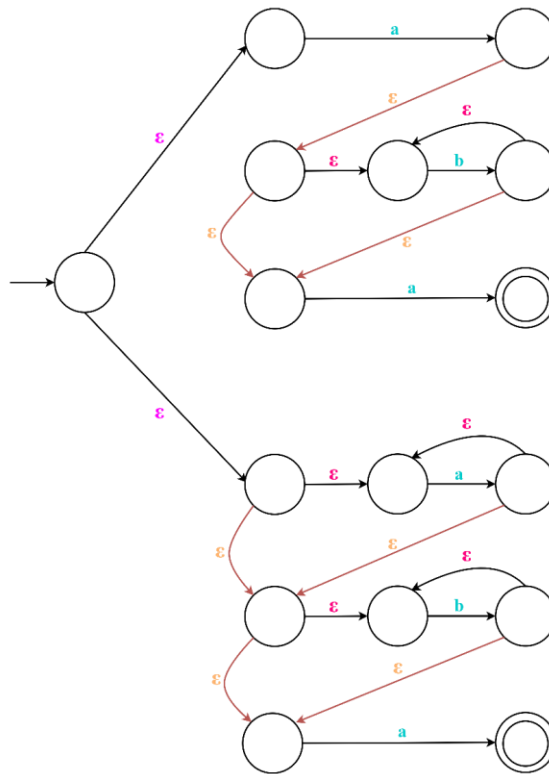
| <p><b>Step 1:</b> Parse the RE into Subexpressions</p> <p>&gt;&gt; <b>The RE “<math>(a b)^* aba</math>” can be decomposed as:</b></p> <ol style="list-style-type: none"> <li>1. <math>(a b)^*</math></li> <li>2. <math>a</math></li> <li>3. <math>b</math></li> <li>4. <math>a</math></li> </ol> <p>We'll <u>build NFAs for each component</u> and then concatenate them.</p> | <p><b>Step 2:</b> Build NFAs for Basic Symbols</p> <table border="1"> <thead> <tr> <th>NFA for a</th><th>NFA for b</th></tr> </thead> <tbody> <tr> <td></td><td></td></tr> </tbody> </table> | NFA for a | NFA for b |  |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-----------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| NFA for a                                                                                                                                                                                                                                                                                                                                                                     | NFA for b                                                                                                                                                                                                                                                                                                                                                         |           |           |                                                                                    |                                                                                     |
|                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                |           |           |                                                                                    |                                                                                     |
| <p><b>Step 4:</b> NFA for <math>(a b)^*</math> (Star)</p>                                                                                                                                                                                                                                   | <p><b>Step 3:</b> NFA for <math>(a b)</math> (Union)</p>                                                                                                                                                                                                                       |           |           |                                                                                    |                                                                                     |
| <p><b>Step 5:</b> NFA for “<math>aba</math>” (Concatenation)</p>                                                                                                                                                                                                                           | <p><b>Step 6:</b> Combine <math>(a b)^*</math> and <math>aba</math> (Concatenation)</p>                                                                                                                                                                                       |           |           |                                                                                    |                                                                                     |



**Problem2:** Convert the following Regular Expression into equivalent NFA using formal procedure. “**ab\*a U a\*b\*a**”.

| <b>Step 1: Parse the RE into Subexpressions</b><br><br>>> The RE “ <b>ab*a U a*b*a</b> ” can be decomposed as:<br>1. <b>ab*a</b><br>2. <b>a*b*a</b><br>3. <b>a</b> 5. <b>a*</b><br>4. <b>b</b> 6. <b>b*</b><br><br>We'll build NFAs for each component and then union them. | <b>Step 2: Build NFAs for Basic Symbols</b> <table><tr><th>NFA for a</th><th>NFA for b</th></tr><tr><td></td><td></td></tr></table> | NFA for a | NFA for b |  |  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-----------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| NFA for a                                                                                                                                                                                                                                                                   | NFA for b                                                                                                                                                                                                                                                                                                |           |           |                                                                                    |                                                                                     |
|                                                                                                                                                                                           |                                                                                                                                                                                                                       |           |           |                                                                                    |                                                                                     |
| <b>Step 4: NFA for b* (Star)</b><br>                                                                                                                                                      | <b>Step 3: NFA for a* (Star)</b><br>                                                                                                                                                                                  |           |           |                                                                                    |                                                                                     |
| <b>Step 5: NFA for ab*a (Concatenation)</b><br>                                                                                                                                          | <b>Step 6: NFA for a*b*a (Concatenation)</b><br>                                                                                                                                                                     |           |           |                                                                                    |                                                                                     |

**Step 7: Final NFA for  $ab^*a \cup a^*b^*a$  (Union)**



 **Problem3:** Convert the following Regular Expression into equivalent NFA using formal procedure. “ $((a | b)^*c)^*$ ”.

| Step 1: Parse the RE into Subexpressions                                                                                                                                                                                                                          | Step 2: Build NFAs for Basic Symbols                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <p>&gt;&gt; The RE “<math>((a b)^*c)^*</math>” can be decomposed as:</p> <ol style="list-style-type: none"> <li>1. <math>(a b)^*</math></li> <li>2. a</li> <li>3. b</li> <li>4. c</li> </ol> <p>We'll <u>build NFAs for each component</u> and then add them.</p> | <div data-bbox="1101 1396 1328 1501"> <p>NFA for a<br/>NFA for b<br/>NFA for c</p> </div> <div data-bbox="998 1522 1339 1827"> </div> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Step 4: NFA for <math>(a b)^*</math> (Star)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | <b>Step 3: NFA for <math>a b</math> (Union)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <p>The diagram shows an NFA for the regular expression <math>(a b)^*</math>. It starts with a start state (double circle) that transitions via <math>\epsilon</math> to a central state. From this central state, there are two paths: one labeled 'a' leading to an accept state (double circle), and another labeled 'b' leading to an accept state. Both paths return to the central state via <math>\epsilon</math> transitions, forming a loop.</p>                                                                                                                                   | <p>The diagram shows an NFA for the regular expression <math>a b</math>. It starts with a start state (double circle) that branches via <math>\epsilon</math> transitions into two parallel paths. The top path consists of a state followed by a transition labeled 'a' to an accept state (double circle). The bottom path consists of a state followed by a transition labeled 'b' to an accept state (double circle).</p>                                                                                                                                                                                                                                                                                                                  |
| <b>Step 5: NFA for <math>(a b)^*c</math> (Concatenation)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | <b>Step 6: Combine <math>((a b)^*c)^*</math>. (Star)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <p>The diagram shows an NFA for the regular expression <math>(a b)^*c</math>. It starts with a start state (double circle) that transitions via <math>\epsilon</math> to a central state. From this central state, there are two paths: one labeled 'a' leading to an accept state (double circle), and another labeled 'b' leading to an accept state (double circle). Both paths return to the central state via <math>\epsilon</math> transitions, forming a loop. Additionally, there is a direct path from the central state to a final accept state (double circle) labeled 'c'.</p> | <p>The diagram shows an NFA for the regular expression <math>((a b)^*c)^*</math>. It starts with a start state (double circle) that transitions via <math>\epsilon</math> to a central state. From this central state, there are two paths: one labeled 'a' leading to an accept state (double circle), and another labeled 'b' leading to an accept state (double circle). Both paths return to the central state via <math>\epsilon</math> transitions, forming a loop. Additionally, there is a direct path from the central state to a final accept state (double circle) labeled 'c'. Finally, there is a long <math>\epsilon</math> transition from the final accept state back to the central state, completing the star operation.</p> |

 **Problem4:** Convert the following Regular Expression into equivalent NFA using formal procedure. “**a (b | c)\*d**”

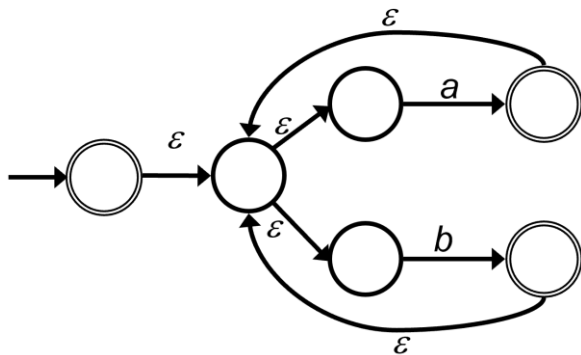
|                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Step 1: Parse the RE into Subexpressions</b>                                                                                                                                                                                                                                | <b>Step 2: Build NFAs for Basic Symbols</b>                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                               |
| <p>&gt;&gt; The RE “<b>a (b   c)*d</b>” can be decomposed as:</p> <ol style="list-style-type: none"> <li>1. <math>(b c)</math></li> <li>2. a</li> <li>3. b</li> <li>4. c</li> <li>5. d</li> </ol> <p>We'll <u>build NFAs for each component</u> and then concatenate them.</p> | NFA for a<br>NFA for b                                                                                                                                                                                                                                                                                                        | NFA for c<br>NFA for d                                                                                                                                                                                                                                                                                                        |
|                                                                                                                                                                                                                                                                                | <p>Two small NFA diagrams are shown. The first is for 'a', consisting of a start state (double circle) and a single transition labeled 'a' to an accept state (double circle). The second is for 'b', consisting of a start state (double circle) and a single transition labeled 'b' to an accept state (double circle).</p> | <p>Two small NFA diagrams are shown. The first is for 'c', consisting of a start state (double circle) and a single transition labeled 'c' to an accept state (double circle). The second is for 'd', consisting of a start state (double circle) and a single transition labeled 'd' to an accept state (double circle).</p> |

|                                                              |                                                               |
|--------------------------------------------------------------|---------------------------------------------------------------|
| <b>Step 4: NFA for <math>(b c)^*</math> (Star)</b>           | <b>Step 3: NFA for <math>(b c)</math> (Union)</b>             |
|                                                              |                                                               |
| <b>Step 5: NFA for <math>(b c)^*d</math> (Concatenation)</b> | <b>Step 6: Combine <math>a(b c)^*d</math> (Concatenation)</b> |
|                                                              |                                                               |

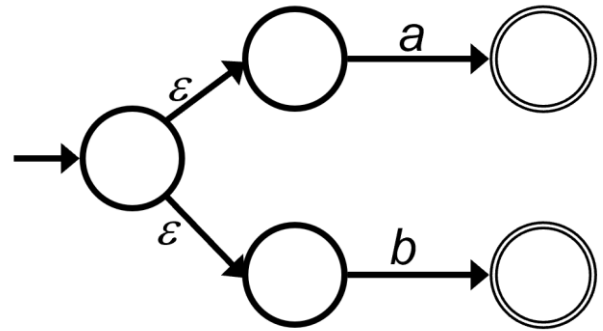
**Problem5:** Convert the following Regular Expression into equivalent NFA using formal procedure.  **$(a | b)^*abc$** .

|                                                                                                                                                                                                                                                                                                                     |                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <b>Step 1: Parse the RE into Subexpressions</b>                                                                                                                                                                                                                                                                     | <b>Step 2: Build NFAs for Basic Symbols</b>          |
| <p>&gt;&gt; <b>The RE <math>(a b)^*abc</math> can be decomposed as:</b></p> <ol style="list-style-type: none"> <li>1. <math>(a b)^*</math></li> <li>2. <math>a</math></li> <li>3. <math>b</math></li> <li>4. <math>c</math></li> </ol> <p>We'll <u>build NFAs for each component</u> and then concatenate them.</p> | <div> NFA for a<br/> NFA for b<br/> NFA for c </div> |

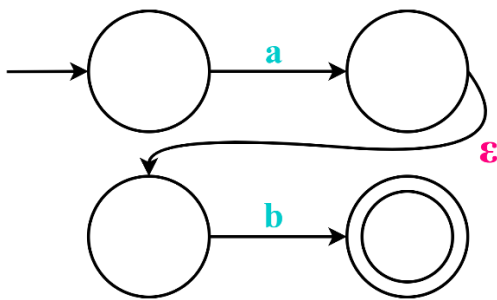
### Step 4: NFA for $(a|b)^*$ (Star)



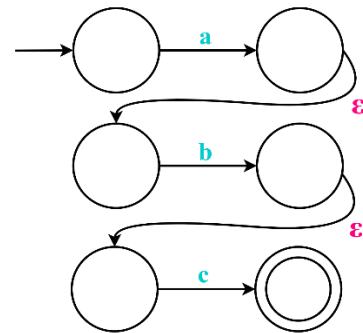
### Step 3: NFA for $(a|b)$ (Union)



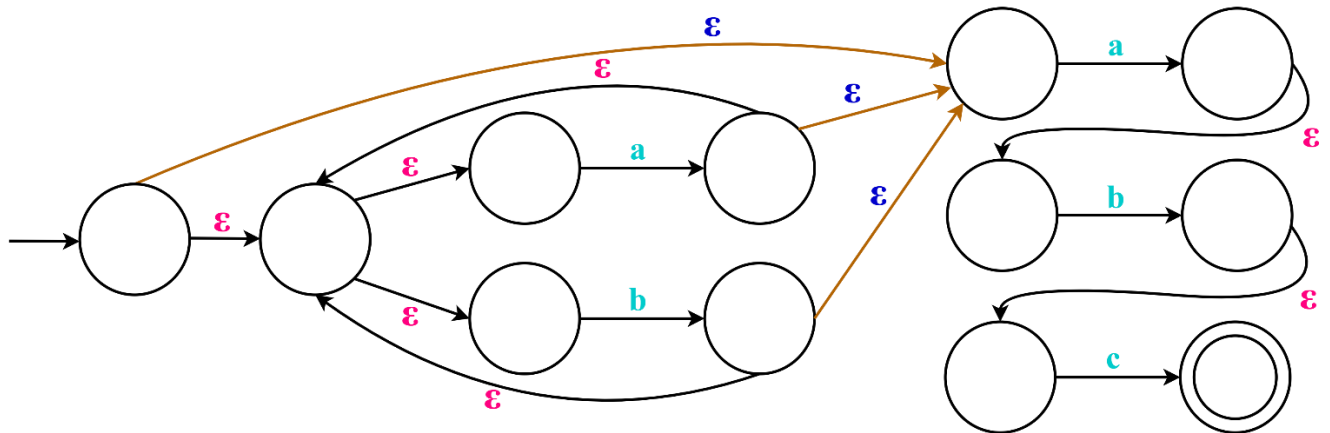
### Step 5: NFA for "ab" (Concatenation)



**Step 6:** Combine “**abc**”. (Concatenation)

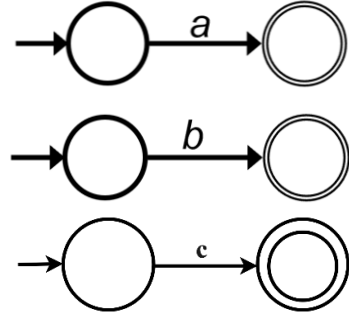
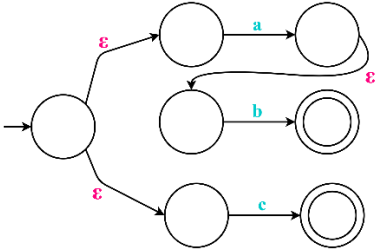
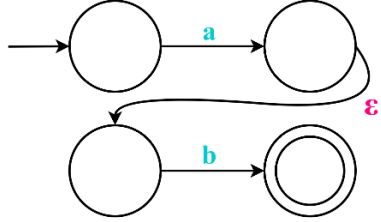
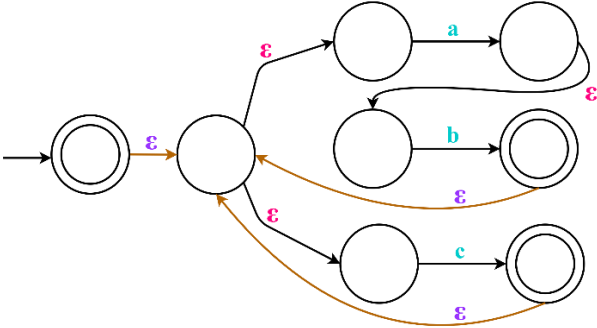
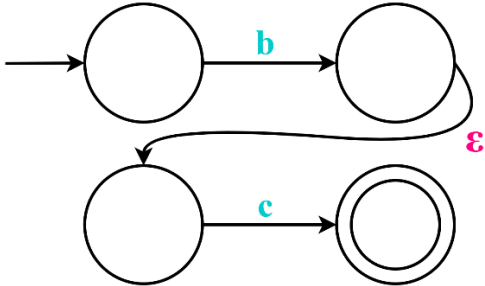


**Step 7:** Combine “(a | b)\*abc”. (Concatenation)

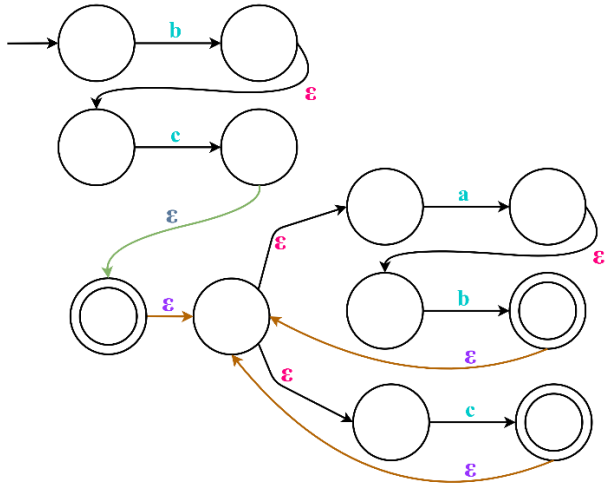




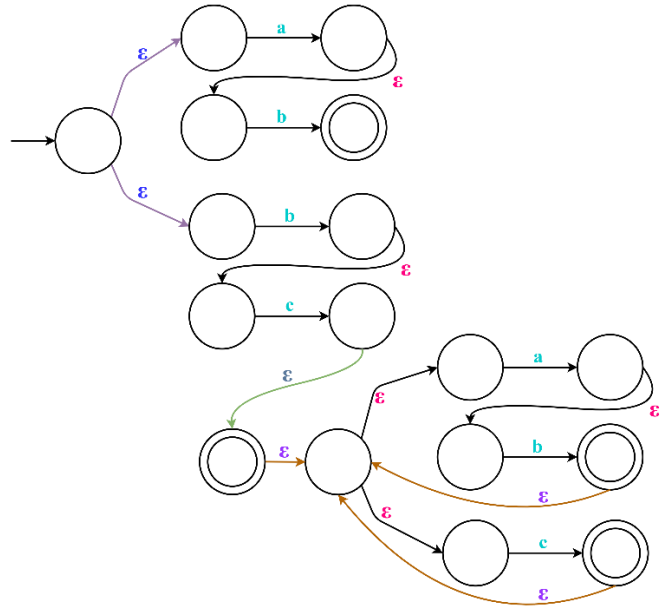
**Problem6:** Convert the following Regular Expression into equivalent NFA using formal procedure. **“(abUbc(abUc)\*)\*”**.

| Step 1: Parse the RE into Subexpressions                                                                                                                                                                                                                   | Step 2: Build NFAs for Basic Symbols                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <p>&gt;&gt; The RE “(abUbc(abUc)*)*” can be decomposed as:</p> <ol style="list-style-type: none"><li>1. (ab c)*</li><li>2. ab</li><li>3. bc</li><li>4. a</li><li>5. b</li><li>6. c</li></ol> <p>We’ll build NFAs for each component and then add them.</p> | <p>NFA for a<br/>NFA for b<br/>NFA for c</p>  |
| Step 4: NFA for (ab c) (Union)                                                                                                                                                                                                                             | Step 3: NFA for (ab) (Concatenation)                                                                                            |
|                                                                                                                                                                          |                                              |
| Step 5: NFA for “(ab c)*” (Star)                                                                                                                                                                                                                           | Step 6: Combine “bc”. (Concatenation)                                                                                           |
|                                                                                                                                                                         |                                             |

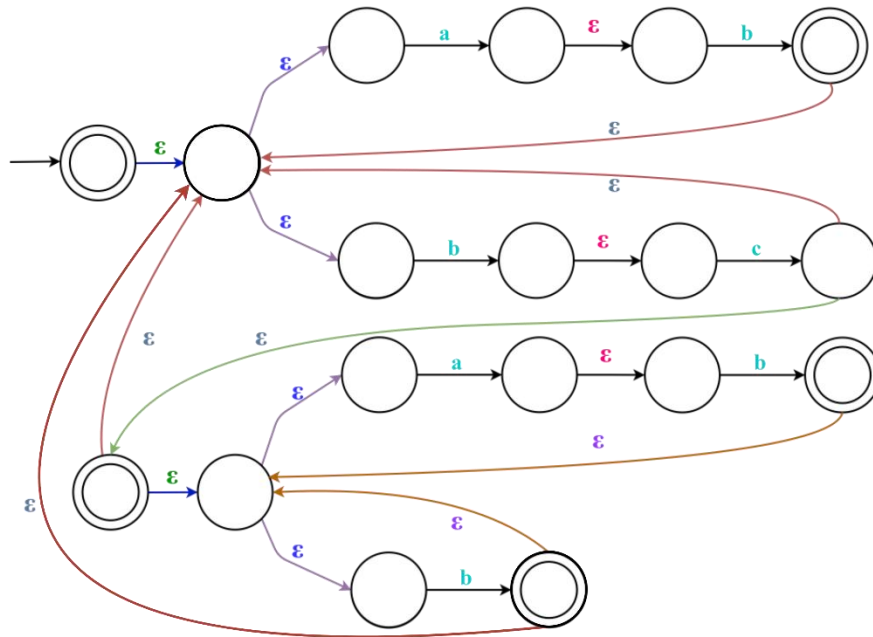
**Step 7:** Combine " $bc(abUc)^*$ ". (Concatenation)



**Step 8:** Combine " $abUbc(abUc)^*$ ". (union)



**Step 9:** Combine " $(abUbc(abUc)^*)^*$ ". (Star)



# CONVERSION OF NFA TO DFA

## Conversion from NFA with $\epsilon$ to DFA

→ Non-deterministic finite automata (NFA) is a finite automata where for some cases when a **specific input is given to the current state**, the machine **goes to multiple states** or **more than 1 states**. It can **contain  $\epsilon$  move**.

It can be represented as  $M = \{Q, \Sigma, \delta, q_0, F\}$ .

Where,

1.  $Q$ : finite set of states
2.  $\Sigma$ : finite set of the input symbol
3.  $q_0$ : initial state
4.  $F$ : final state
5.  $\delta$ : Transition function

NFA with  $\epsilon$  move: If any FA **contains  $\epsilon$  transaction or move**, the finite automata is called NFA with  $\epsilon$  move.

$\epsilon$ -closure:  $\epsilon$ -closure for a given **state A** means a **set of states** which can be **reached from the state A with only  $\epsilon$ (null) move** including the state A itself.

## Steps for converting NFA with $\epsilon$ to DFA

**Step 1:** We will take the  $\epsilon$ -closure for the starting state of NFA as a starting state of DFA.

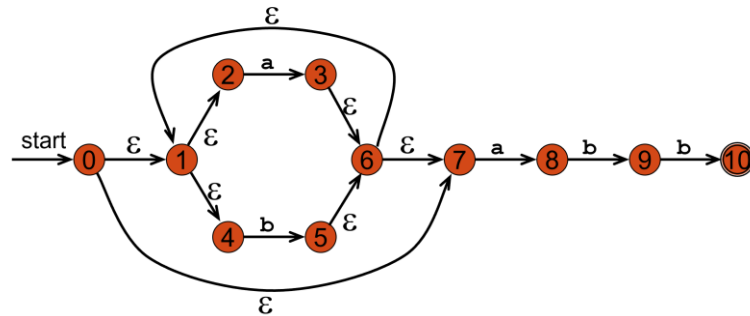
**Step 2:** Find the states for each input symbol that can be traversed from the present. That means the union of transition value and their closures for each state of NFA present in the current state of DFA.

**Step 3:** If we found a new state, take it as current state and repeat step 2.

**Step 4:** Repeat Step 2 and Step 3 until there is no new state present in the transition table of DFA.

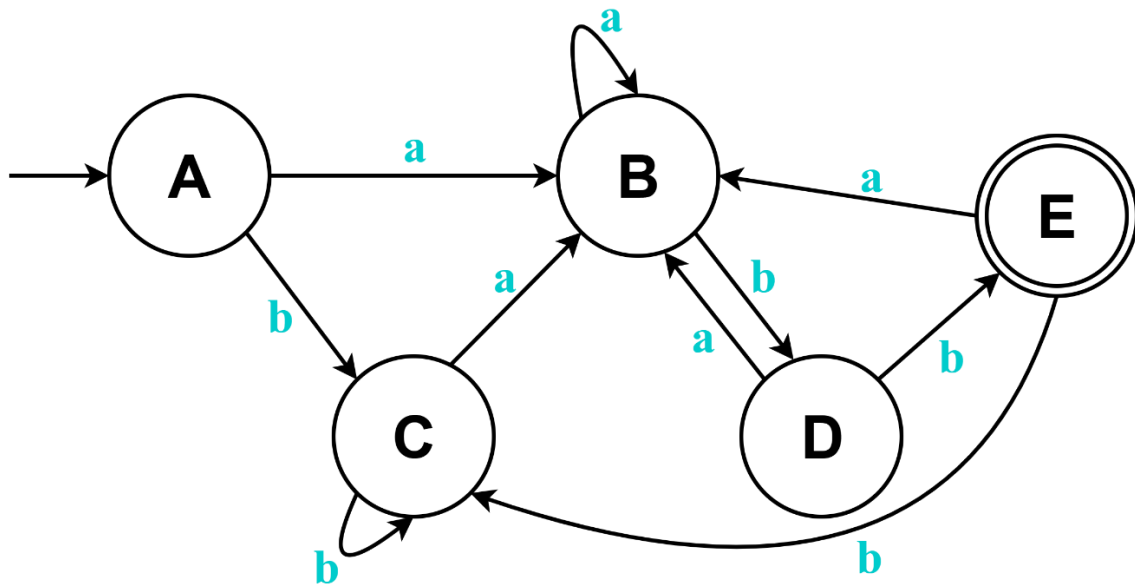
**Step 5:** Mark the states of DFA as a final state which contains the final state of NFA.

🔗 **Problem1:** Convert the following NFA with  $\epsilon$  into equivalent DFA using Subset Construction Method



| $\epsilon$ -Closure Table for DFA States |                                     |                                    | DFA Transition Table Constructed from NFA |                 |   |   |
|------------------------------------------|-------------------------------------|------------------------------------|-------------------------------------------|-----------------|---|---|
| DFA State                                | $\epsilon$ -closure of              | $\epsilon$ -closure outcome states | NFA States                                | DFA State       | a | b |
| A                                        | $\epsilon$ -Closure of $(\{0\})$    | 0,1,2,4,7                          | 0,1,2,4,7                                 | $\rightarrow$ A | B | C |
| B                                        | $\epsilon$ -Closure of $(\{3,8\})$  | 3,6,7,1,2,4,8                      | 3,6,7,1,2,4,8                             | B               | B | D |
| C                                        | $\epsilon$ -Closure of $(\{5\})$    | 5,6,7,1,2,4                        | 5,6,7,1,2,4                               | C               | B | C |
| D                                        | $\epsilon$ -Closure of $(\{5,9\})$  | 5,6,7,1,2,4,9                      | 5,6,7,1,2,4,9                             | D               | B | E |
| E                                        | $\epsilon$ -Closure of $(\{5,10\})$ | 5,6,7,1,2,4,10                     | 5,6,7,1,2,4,10                            | E               | B | C |

Now, Construct the DFA from [transition table](#):



## "An Alternative Approach to Solving It"

**Step1:** Write the transition table of NFA

| $\delta$ | a      | b      |
|----------|--------|--------|
| 0        | $\phi$ | $\phi$ |
| 1        | $\phi$ | $\phi$ |
| 2        | 3      | $\phi$ |
| 3        | $\phi$ | $\phi$ |
| 4        | $\phi$ | 5      |
| 5        | $\phi$ | $\phi$ |
| 6        | $\phi$ | $\phi$ |
| 7        | 8      | $\phi$ |
| 8        | $\phi$ | 9      |
| 9        | $\phi$ | 10     |
| 10       | $\phi$ | $\phi$ |

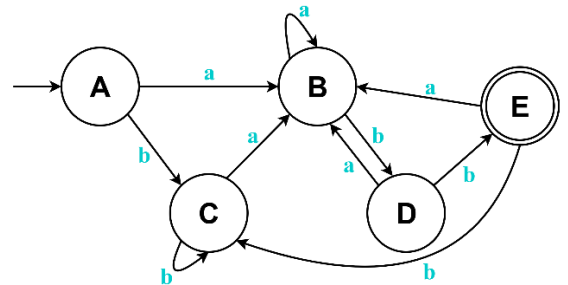
**Step2:** Find the Epsilon ( $\epsilon$ ) Closure of all the state

- $\epsilon$ -Closure of  $(\{0\}) = \{0, 1, 2, 4, 7\}$
- $\epsilon$ -Closure of  $(\{1\}) = \{1, 2, 4\}$
- $\epsilon$ -Closure of  $(\{2\}) = \{2\}$
- $\epsilon$ -Closure of  $(\{3\}) = \{3, 6, 7, 1, 2, 4\}$
- $\epsilon$ -Closure of  $(\{4\}) = \{4\}$
- $\epsilon$ -Closure of  $(\{5\}) = \{5, 6, 7, 1, 2, 4\}$
- $\epsilon$ -Closure of  $(\{6\}) = \{6, 7, 1, 2, 4\}$
- $\epsilon$ -Closure of  $(\{7\}) = \{7\}$
- $\epsilon$ -Closure of  $(\{8\}) = \{8\}$
- $\epsilon$ -Closure of  $(\{9\}) = \{9\}$
- $\epsilon$ -Closure of  $(\{10\}) = \{10\}$

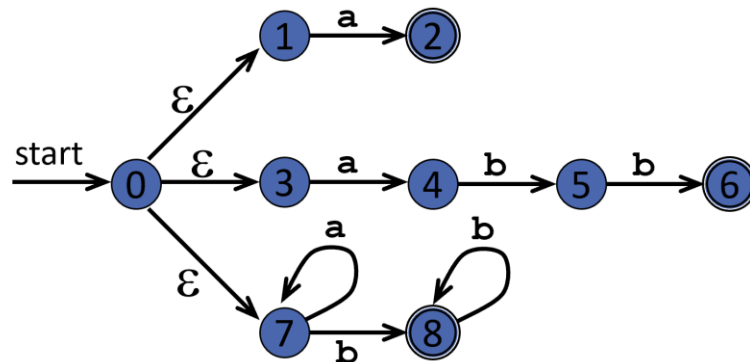
**Step3:** Convert this NFA transition table to its equivalent DFA transition table.

| $\epsilon$ -closure of              | NFA            | DFA | a | b |
|-------------------------------------|----------------|-----|---|---|
| $\epsilon$ -Closure of $(\{0\})$    | 0,1,2,4,7 → A  | A   | B | C |
| $\epsilon$ -Closure of $(\{3,8\})$  | 3,6,7,1,2,4,8  | B   | B | D |
| $\epsilon$ -Closure of $(\{5\})$    | 5,6,7,1,2,4    | C   | B | C |
| $\epsilon$ -Closure of $(\{5,9\})$  | 5,6,7,1,2,4,9  | D   | B | E |
| $\epsilon$ -Closure of $(\{5,10\})$ | 5,6,7,1,2,4,10 | E   | B | C |

**Step4:** Now, Construct the DFA from transition table.



🔗 **Problem2:** Convert the following NFA with  $\epsilon$  into equivalent DFA using Subset Construction Method



**Step1:** Write the transition table of NFA

| $\delta$ | a      | b      |
|----------|--------|--------|
| 0        | $\phi$ | $\phi$ |
| 1        | 2      | $\phi$ |
| 2        | $\phi$ | $\phi$ |
| 3        | 4      | $\phi$ |
| 4        | $\phi$ | 5      |
| 5        | $\phi$ | 6      |
| 6        | $\phi$ | $\phi$ |
| 7        | 7      | 8      |
| 8        | $\phi$ | 8      |

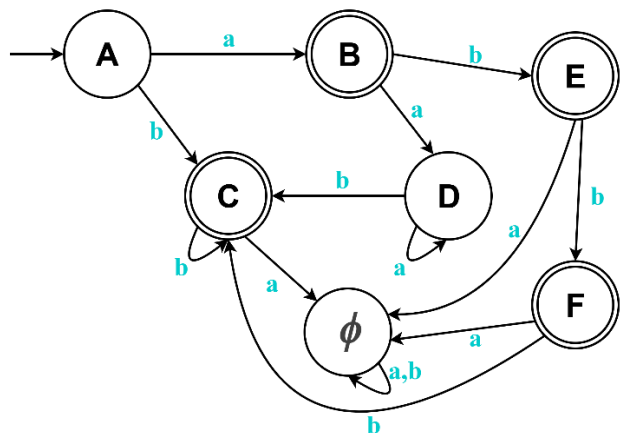
**Step2:** Find the Epsilon ( $\epsilon$ ) Closure of all the state

- $\epsilon$ -Closure of  $(\{0\}) = \{0, 1, 3, 7\}$
- $\epsilon$ -Closure of  $(\{1\}) = \{1\}$
- $\epsilon$ -Closure of  $(\{2\}) = \{2\}$
- $\epsilon$ -Closure of  $(\{3\}) = \{3\}$
- $\epsilon$ -Closure of  $(\{4\}) = \{4\}$
- $\epsilon$ -Closure of  $(\{5\}) = \{5\}$
- $\epsilon$ -Closure of  $(\{6\}) = \{6\}$
- $\epsilon$ -Closure of  $(\{7\}) = \{7\}$
- $\epsilon$ -Closure of  $(\{8\}) = \{8\}$

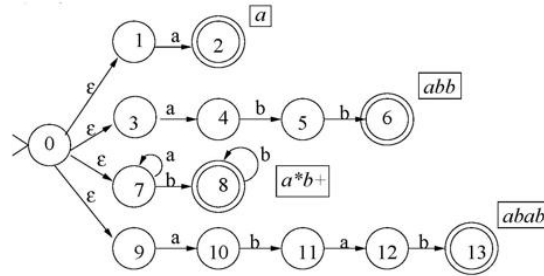
**Step3:** Convert this NFA transition table to its equivalent DFA transition table.

| $\epsilon$ -closure of               | NFA     | DFA    | a      | b      |
|--------------------------------------|---------|--------|--------|--------|
| $\epsilon$ -Closure of $(\{0\})$     | 0,1,3,7 | A      | B      | C      |
| $\epsilon$ -Closure of $(\{2,4,7\})$ | 2,4,7   | B      | D      | E      |
| $\epsilon$ -Closure of $(\{8\})$     | 8       | C      | $\phi$ | C      |
| $\epsilon$ -Closure of $(\{7\})$     | 7       | D      | D      | C      |
| $\epsilon$ -Closure of $(\{5,8\})$   | 5,8     | E      | $\phi$ | F      |
| $\epsilon$ -Closure of $(\{6,8\})$   | 6,8     | F      | $\phi$ | C      |
|                                      |         | $\phi$ | $\phi$ | $\phi$ |

**Step4:** Now, Construct the DFA from transition table.



✂ **Problem2:** Convert the following NFA with  $\epsilon$  into equivalent DFA using Subset Construction Method



**Step1:** Write the transition table of NFA

| $\delta$ | a      | b      |
|----------|--------|--------|
| 0        | $\phi$ | $\phi$ |
| 1        | 2      | $\phi$ |
| 2        | $\phi$ | $\phi$ |
| 3        | 4      | $\phi$ |
| 4        | $\phi$ | 5      |
| 5        | $\phi$ | 6      |
| 6        | $\phi$ | $\phi$ |
| 7        | 7      | 8      |
| 8        | $\phi$ | 8      |
| 9        | 10     | $\phi$ |
| 10       | $\phi$ | 11     |
| 11       | 12     | $\phi$ |
| 12       | $\phi$ | 13     |
| 13       | $\phi$ | $\phi$ |

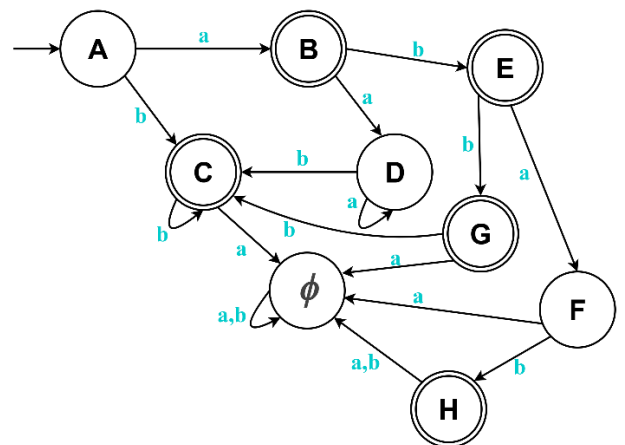
**Step2:** Find the Epsilon ( $\epsilon$ ) Closure of all the state

- $\epsilon$ -Closure of  $(\{0\}) = \{0, 1, 3, 7, 9\}$
- $\epsilon$ -Closure of  $(\{1\}) = \{1\}$
- $\epsilon$ -Closure of  $(\{2\}) = \{2\}$
- $\epsilon$ -Closure of  $(\{3\}) = \{3\}$
- $\epsilon$ -Closure of  $(\{4\}) = \{4\}$
- $\epsilon$ -Closure of  $(\{5\}) = \{5\}$
- $\epsilon$ -Closure of  $(\{6\}) = \{6\}$
- $\epsilon$ -Closure of  $(\{7\}) = \{7\}$
- $\epsilon$ -Closure of  $(\{8\}) = \{8\}$
- $\epsilon$ -Closure of  $(\{9\}) = \{9\}$
- $\epsilon$ -Closure of  $(\{10\}) = \{10\}$
- $\epsilon$ -Closure of  $(\{11\}) = \{11\}$
- $\epsilon$ -Closure of  $(\{12\}) = \{12\}$
- $\epsilon$ -Closure of  $(\{13\}) = \{13\}$

**Step3:** Convert this NFA transition table to its equivalent DFA transition table.

| $\epsilon$ -closure of                  | NFA       | DFA | a      | b      |
|-----------------------------------------|-----------|-----|--------|--------|
| $\epsilon$ -Closure of $(\{0\})$        | 0,1,3,7,9 | A   | B      | C      |
| $\epsilon$ -Closure of $(\{2,4,7,10\})$ | 2,4,7,10  | B   | D      | E      |
| $\epsilon$ -Closure of $(\{8\})$        | 8         | C   | $\phi$ | C      |
| $\epsilon$ -Closure of $(\{7\})$        | 7         | D   | D      | C      |
| $\epsilon$ -Closure of $(\{5,8,11\})$   | 5,8,11    | E   | F      | G      |
| $\epsilon$ -Closure of $(\{12\})$       | 12        | F   | $\phi$ | H      |
| $\epsilon$ -Closure of $(\{6,8\})$      | 6,8       | G   | $\phi$ | C      |
| $\epsilon$ -Closure of $(\{13\})$       | 13        | H   | $\phi$ | $\phi$ |

**Step4:** Now, Construct the DFA from transition table.

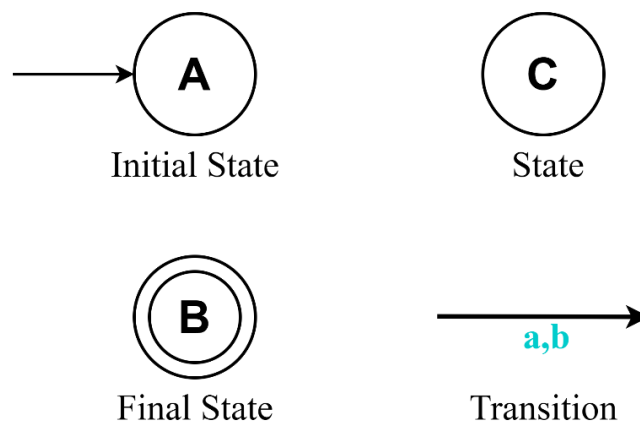


# **DETERMINISTIC FINITE AUTOMATON (DFA)**

# Finite Representation (FR) / Finite Automaton (FA)

- **Definition:** If a language can be recognized with a finite number of states, it's called Finite Representation (FR) or Finite Automaton (FA).
- **Key Point:** FA uses a finite number of states to represent and accept or reject strings from a language.

## ➔ Finite Representation:

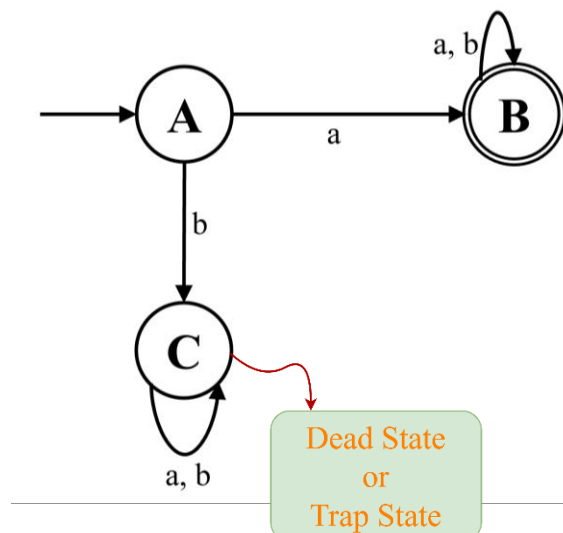


### 🔗 Example 1:

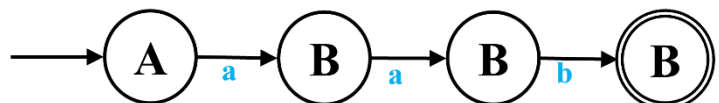
➔  $\Sigma = \{a, b\}$

➔ **L1 = Set of all strings starting with 'a'**

➔ **L1 = {a, aa, ab, abb, aba, ...}**



**ACCEPT:**



**REJECT:**

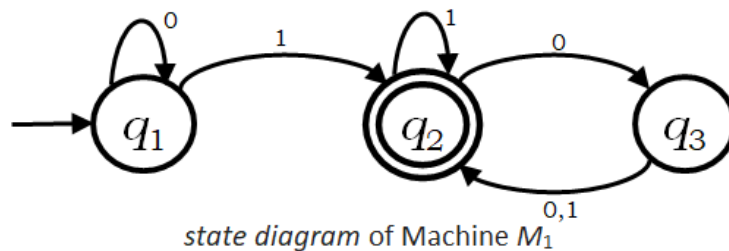


# Formal Definition - DFA

→ A DFA is a 5-tuple:  $(Q, \Sigma, \delta, q_0, F)$

1.  $Q \rightarrow$  Finite set of states.
2.  $\Sigma \rightarrow$  Finite input alphabet.
3.  $\delta: Q \times \Sigma \rightarrow Q \rightarrow$  Transition function (maps state & input to next state).
4.  $q_0 \in Q \rightarrow$  Start state.
5.  $F \subseteq Q \rightarrow$  Set of accept (final) states.

✂ Example 1:



→ Here,

➤  $M_1 = (Q, \Sigma, d, q_0, F),$

➤ Where:

$$Q = \{q_1, q_2, q_3\},$$

$$\Sigma = \{0, 1\},$$

$$F = \{q_2\},$$

$$q_0 = \{q_1\},$$

*$\delta$  is describe as:*

$$\begin{aligned} \delta(q_1, 0) &= q_1, & \delta(q_1, 1) &= q_2, \\ \delta(q_2, 0) &= q_3, & \delta(q_2, 1) &= q_2, \\ \delta(q_3, 0) &= q_2, & \delta(q_3, 1) &= q_2. \end{aligned}$$

Transition Function

or

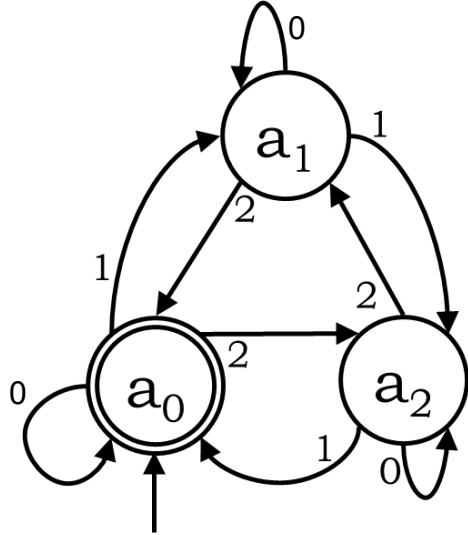
| $\delta$ | 0     | 1     |
|----------|-------|-------|
| $q_1$    | $q_1$ | $q_2$ |
| $q_2$    | $q_3$ | $q_2$ |
| $q_3$    | $q_2$ | $q_2$ |

Transition Table

# Designing DFA

## Example 1:

→ Design a DFA that accepts strings where the sum of all symbols (0,1,2) is a multiple of 3.



Input example: 01120101

Input symbol: Accepted

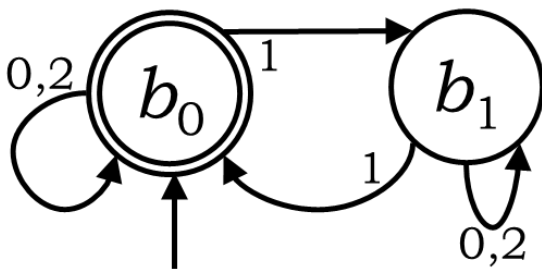
### → DFA for Sum of Symbols $\equiv 0,1,2 \pmod 3$ :

- **Alphabet:**  $\Sigma = \{0,1,2\}$
- **Language:** Strings where the sum of symbols is a multiple of 3.
- **States:**  $Q = \{a_0, a_1, a_2\}$ , where  $a_i$  represents  $S \bmod 3 = i$ .
- **Start State:**  $q_0 = a_0$
- **Final State:**  $F = \{a_0\}$
- **Transition Table ( $\delta_1$ ):**

| $\delta$ | 0     | 1     | 2     |
|----------|-------|-------|-------|
| $a_0$    | $a_0$ | $a_1$ | $a_2$ |
| $a_1$    | $a_1$ | $a_2$ | $a_0$ |
| $a_2$    | $a_2$ | $a_0$ | $a_1$ |

## Example 2:

→ Design a DFA that accepts strings where the sum of all symbols (0,1,2) is an even number.



Input example: 01120101

Input symbol: Accepted

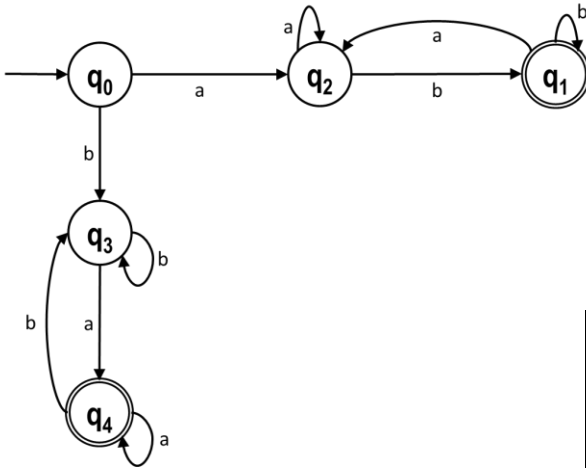
### DFA for Sum of Symbols $\equiv 0,1,2 \pmod 2$ (Even Sum):

- **Alphabet:**  $\Sigma = \{0,1,2\}$
- **Language:** Strings where the sum of symbols is an even number.
- **States:**  $Q = \{b_0, b_1\}$ , where  $b_i$  represents  $S \bmod 2 = i$ .
- **Start State:**  $q_0 = b_0$
- **Final State:**  $F = \{b_0\}$
- **Transition Table ( $\delta_2$ ):**

| $\delta$ | 0     | 1     | 2     |
|----------|-------|-------|-------|
| $b_0$    | $b_0$ | $b_1$ | $b_0$ |
| $b_1$    | $b_1$ | $b_0$ | $b_1$ |

### Example 3:

→ Identify the Components of the DFA



### Components of the DFA:

- **Alphabet:**  $\Sigma = \{a, b\}$
- **States:**  $Q = \{q_0, q_1, q_2, q_3, q_4\}$ ,
- **Start State:**  $q_0 = q_0$
- **Final State:**  $F = \{q_1, q_4\}$
- **Transition Table ( $\delta$ ):**

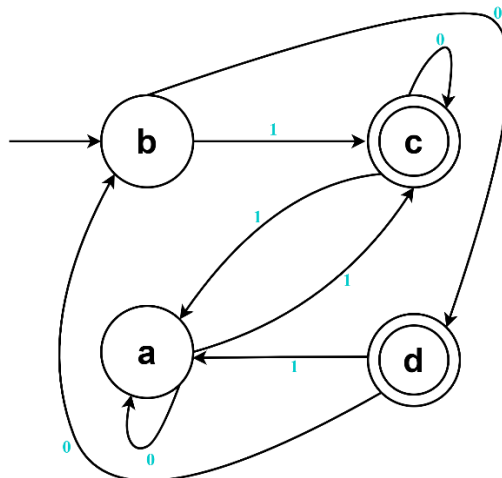
| $\delta$ | a     | b     |
|----------|-------|-------|
| $q_0$    | $q_2$ | $q_3$ |
| $q_1$    | $q_2$ | $q_1$ |
| $q_2$    | $q_2$ | $q_1$ |
| $q_3$    | $q_4$ | $q_3$ |
| $q_4$    | $q_4$ | $q_3$ |

### Example 4:

→ Design a DFA that  $Q = \{a, b, c, d\}$ ,  $\Sigma = \{0, 1\}$ ,  $q_0 = b$ ,  $F = \{c, d\}$ ,

| $\delta$ | 0 | 1 |
|----------|---|---|
| a        | a | c |
| b        | d | c |
| c        | c | a |
| d        | b | a |

→ **Solution:**



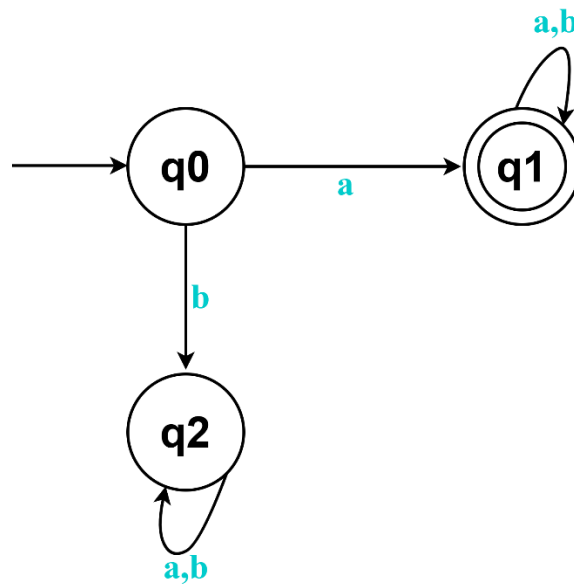
### ✎ Example 5:

→ Given the state diagram of the machine (  $\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_1\}$  ) Where  $\delta$  is given the following table

| $\delta$ | a     | b     |
|----------|-------|-------|
| $q_0$    | $q_1$ | $q_2$ |
| $q_1$    | $q_1$ | $q_1$ |
| $q_2$    | $q_2$ | $q_2$ |

→ Design The DFA

→ **Solution:**

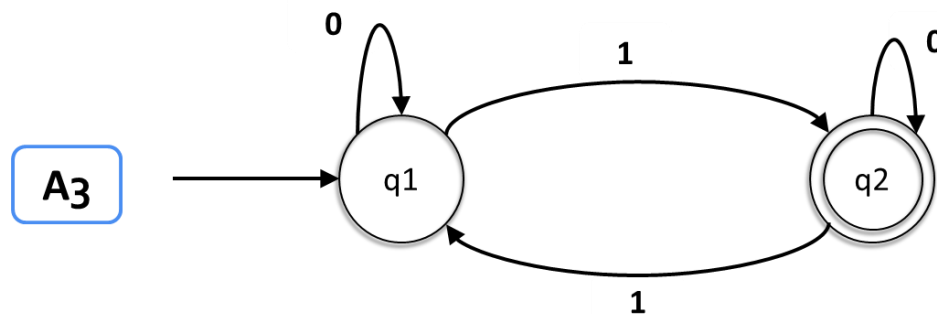


### ✎ Example 6:

→  $A_3 = \{w : w \text{ is a binary string containing an odd number of 1s}\}. \Sigma = \{0,1\}$

→ So,  $w_1 = 1, w_2 = 10, w_3 = 10101, w_4 = 10101\dots$

Now,  $L = \{1, 100, 10101, 11010, 11100\dots\}$



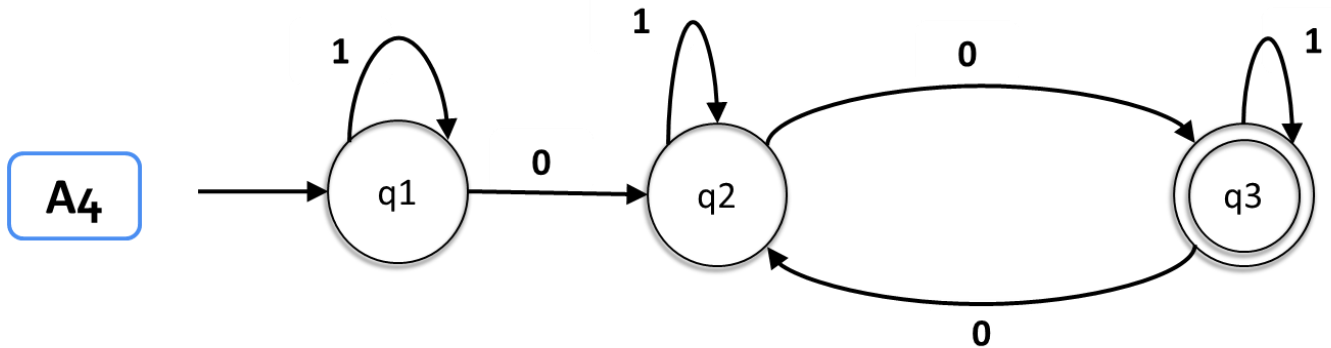
**Accepted**

✂ Example 7:

→  $A_4 = \{w : w \text{ is a binary string containing an even number of } 0\text{s}\}. \Sigma = \{0,1\}$

→ So,  $w_1 = 00, w_2 = 001, w_3 = 0101, w_4 = 0101010...$

Now,  $L = \{00, 100, 1010, 11010, 11100...\}$

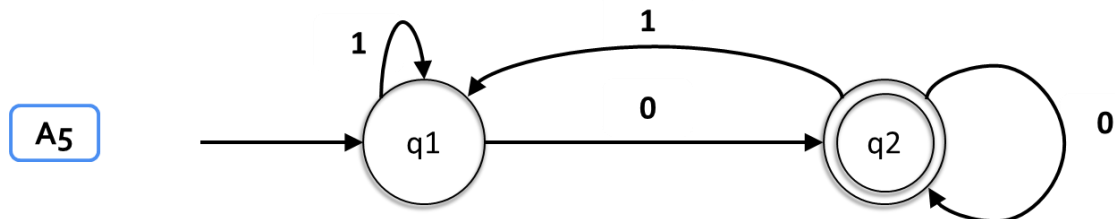


✂ Example 8:

→  $A_5 = \{w \mid w \text{ ends with a } 0\}. \Sigma = \{0,1\}$

→ So,  $w_1 = 010, w_2 = 000, w_3 = 10, w_4 = 0101010...$

Now,  $L = \{0, 10, 100, 11010, 11100...\}$

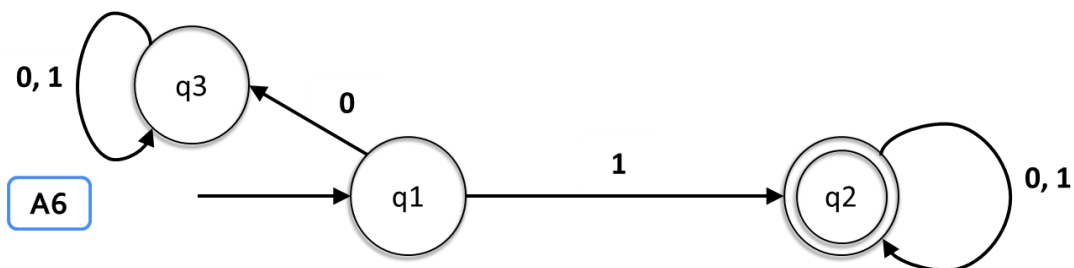


✂ Example 9:

→  $A_6 = \{w \mid w \text{ begins with a } 1\}. \Sigma = \{0,1\}$

→ So,  $w_1 = 100, w_3 = 10, w_5 = 101010...$

Now,  $L = \{1, 11, 101, 1001, 11100...\}$

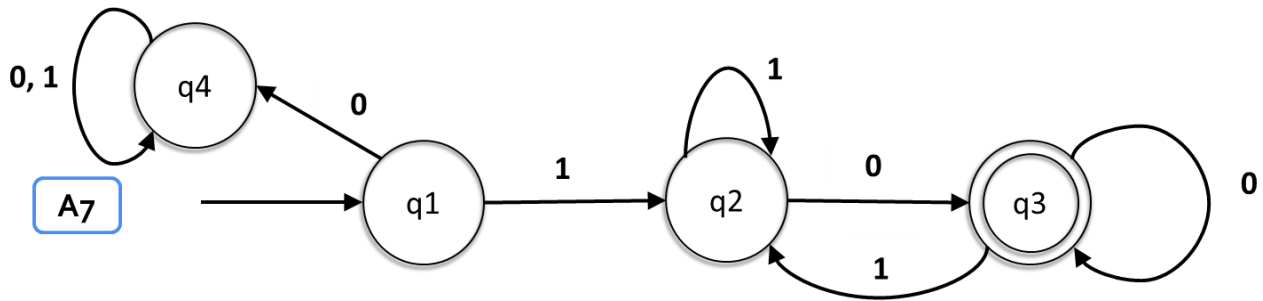


✂ Example 10:

→  $A7 = \{w \mid w \text{ begins with a } 1 \text{ and ends with a } 0\}$ .  $\Sigma = \{0,1\}$

→ So,  $w_1 = 100$ ,  $w_2 = 10$ ,  $w_3 = 101010...$

Now,  $L = \{1, 100, 1010, 11010, 11100...\}$

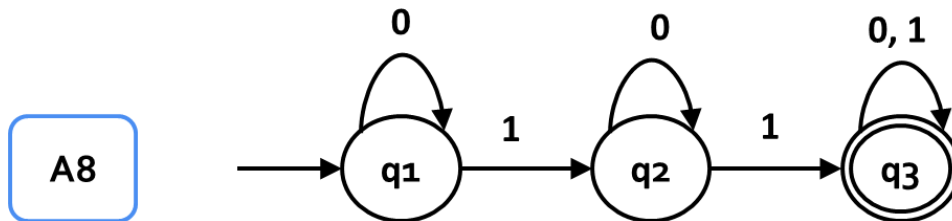


✂ Example 11:

→  $A8 = \{w \mid w \text{ has at least two } 1\text{s}\}$ .  $\Sigma = \{0,1\}$

→ So,  $w_1 = 1010$ ,  $w_2 = 101$ ,  $w_4 = 01010$ ,  $w_5 = 101010...$

Now,  $L = \{11, 1100, 1010, 11010, 11100...\}$

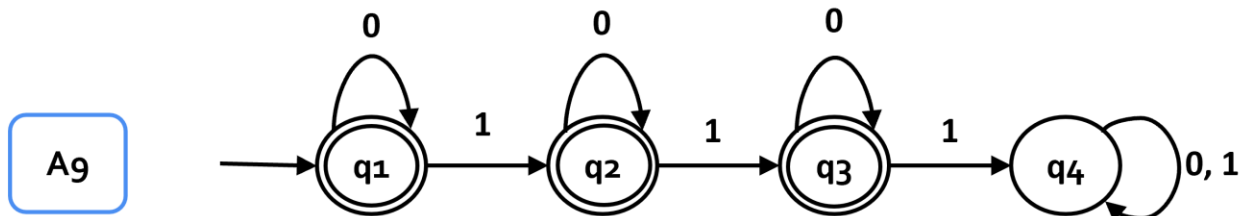


✂ Example 12:

→  $A9 = \{w \mid w \text{ has at most two } 1\text{s}\}$ .  $\Sigma = \{0,1\}$

→ So,  $w_1 = 1010$ ,  $w_2 = 001$ ,  $w_3 = 000$ ,  $w_4 = 10100...$

Now,  $L = \{11, 1100, 1010, 11010, 11100...\}$

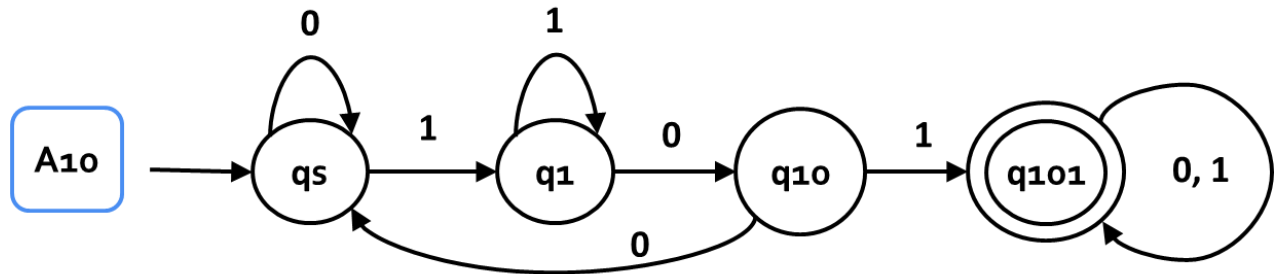


✂ **Example 13:**

→  $A_{10} = \{w \mid w \text{ has substring } 101\}$ .  $\Sigma = \{0,1\}$

→ So,  $w_1 = 1010$ ,  $w_2 = 001$ ,  $w_3 = 10100...$

Now,  $L = \{101, 1101, 10101, 11010, 111010...\}$

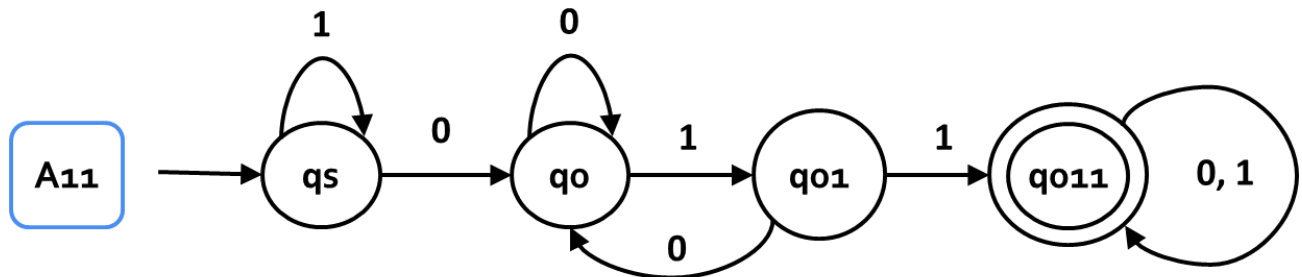


✂ **Example 14:**

→  $A_{11} = \{w \mid w \text{ has substring } 011\}$ .  $\Sigma = \{0,1\}$

→ So,  $w_1 = 10110$ ,  $w_2 = 000110$ ,  $w_3 = 101100...$

Now,  $L = \{011, 11011, 10101011, 110110, 1011010...\}$



→ What happens for the language,  $A_{11} = \{w \mid w \text{ does not have substring } 011\}$ ?

→ For  $A_{11} = \{w \mid w \text{ has substring "011"}\}$ , the DFA **accepts** strings containing "011".

→ For  $A'_{11} = \{w \mid w \text{ does not have substring "011"}\}$ , the DFA **rejects** any string with "011" and stays in states ensuring "011" never appears.

→ A **dead state** is used for rejection.

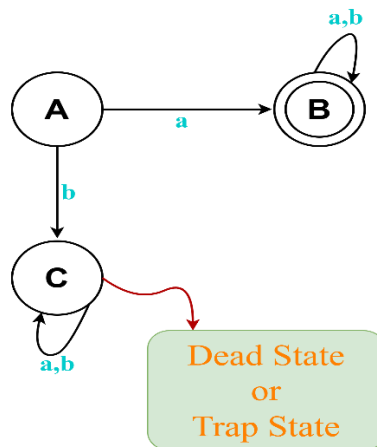
### Problem 1:

>> Construct a DFA which accepts set of all string over the  $\Sigma = \{a,b\}$ , where each string start with an "a"

Or  $L(M) = \{w \mid w \text{ Start with an "a"}\}$

→ Is Given,

$\Sigma = \{a,b\}$ ,  
 $L(M) = \{a, aa, aba, aabba, \dots\}$



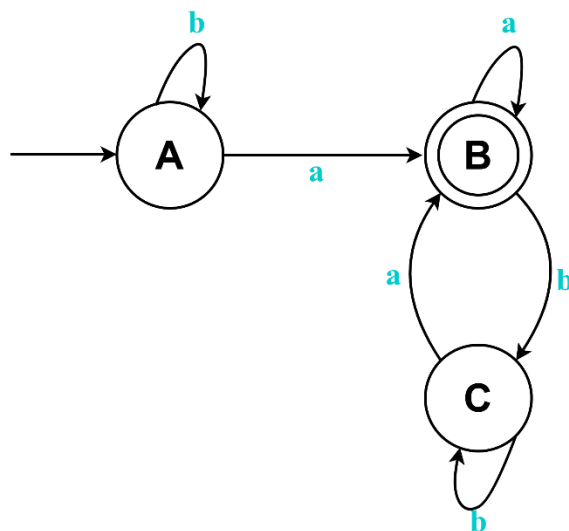
### Problem 2:

>> Construct a DFA which accepts set of all string over the  $\Sigma = \{a,b\}$ , where each string end with an "a"

Or  $L(M) = \{w \mid w \text{ End with an "a"}\}$

→ Is Given,

$\Sigma = \{a,b\}$ ,  
 $L(M) = \{a, aa, aba, aabba, abababa, bba, \dots\}$



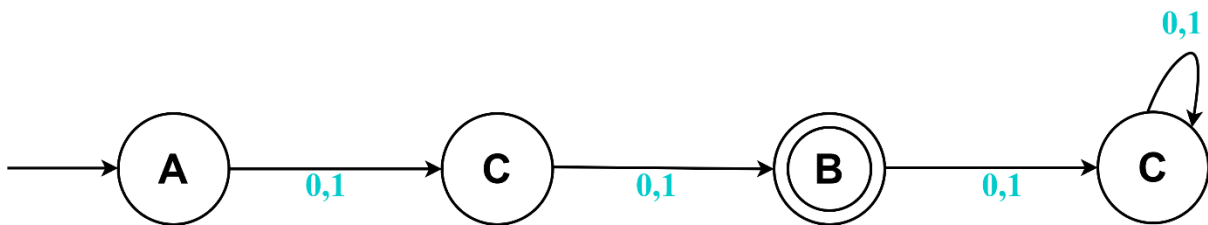
### Problem 3:

» Construct a DFA that accepts set of all string over the  $\Sigma = \{0,1\}$ , of length 2.

→ Is Given,

$$\Sigma = \{0,1\},$$

$$L(M) = \{00,01,10,11\}$$



### Problem 4:

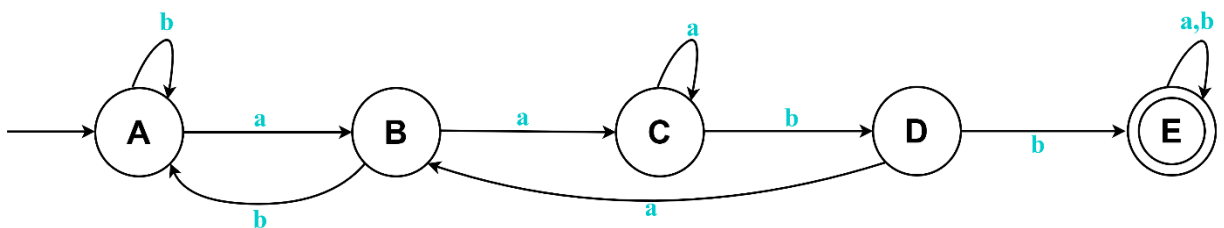
» Construct a DFA that accepts any string over the  $\Sigma = \{a,b\}$ , that does not contain the string “aabb” in it.

→ Is Given,

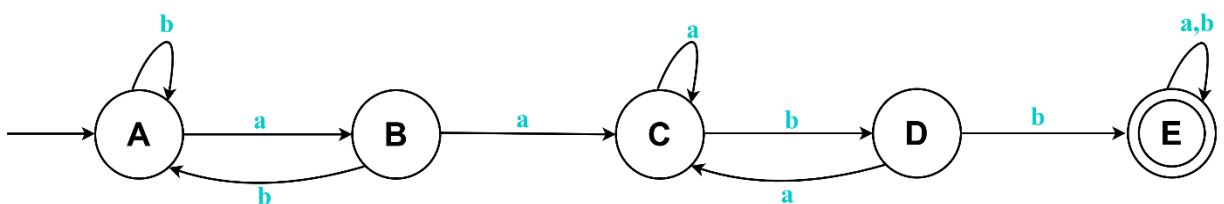
$$\Sigma = \{a,b\},$$

→ Let, Construct a DFA that accepts any string over the  $\Sigma = \{a,b\}$ , that contains the string “aabb” in it.

$$L(M) = \{aabb, baabb, babaabb, babaaabb, aabaabb, aababaabb, \dots\}$$



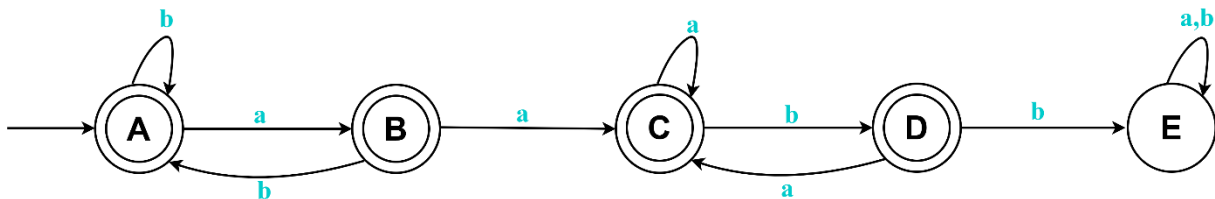
OR



→ Okay, This is not our desired output since it does contain the string "aabb."

→ So, To have our desired output, we just need to:

- Flip the states.
- Make the final states non-final state.
- Make the non-final states final state.



### ✂ Problem 5:

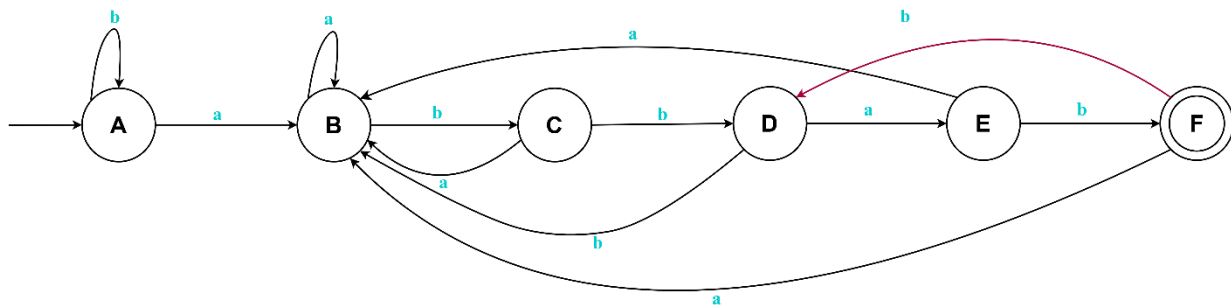
>> Design DFA for the following Language. Where  $\Sigma = \{a,b\}$  and give the formal definition of your machine. (Any-ONE)

- $L = \{w \mid \text{ends with 'abbab' substring}\}$
- $L = \{w \mid w \text{ contains exactly two a's and odd number of b's}\}$  Mode1#

(i)

→ Is Given,

$\Sigma = \{a,b\}$ ,  
 $L(M1) = \{\text{abbab, babbab, aabbab, ababbab, abbbabbab, abbabbab, abbababbab, .....}\}$

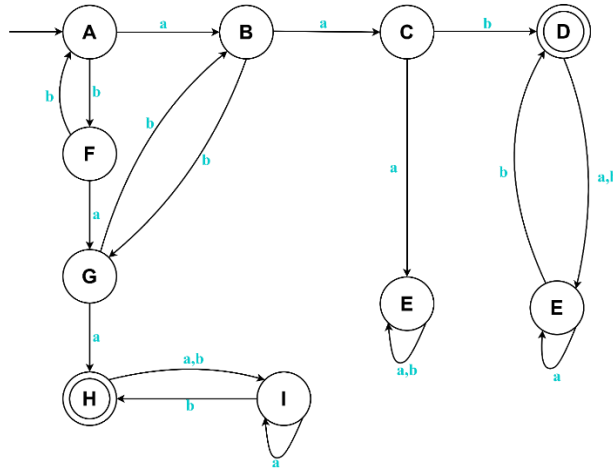


(ii)

→ Is Given,

$\Sigma = \{a,b\}$ ,

$L(M1) = \{baa, aab, baabb, babab, bbababb, bbababb, abbab, \dots\}$



### Problem 6:

>> Design DFA for the following Language. Consider  $\Sigma = \{x,y\}$ ,

i.  $\{w \mid \text{each 'y' in } w \text{ is followed by at least two 'x' }\}$  and simulate your machine using the string: “xyxyxx” and comment whether this string is **accepted/ rejected** by your machine.

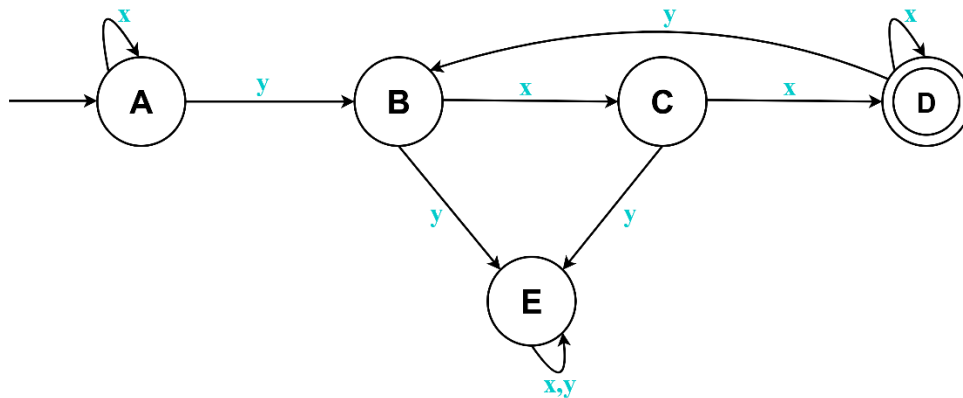
ii.  $\{w \mid w \text{ starts and ends with same symbol}\}$  and simulate your machine using the string: “yyxyxy” and comment whether this string is **accepted/ rejected** by your machine.

(i)

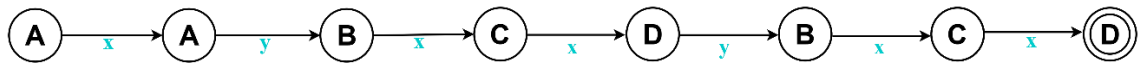
→ Is Given,

$\Sigma = \{x,y\}$ ,

$L(M1) = \{yxx, yxyxx, yxxxx, \dots\}$



Eg: xyxxxyxx



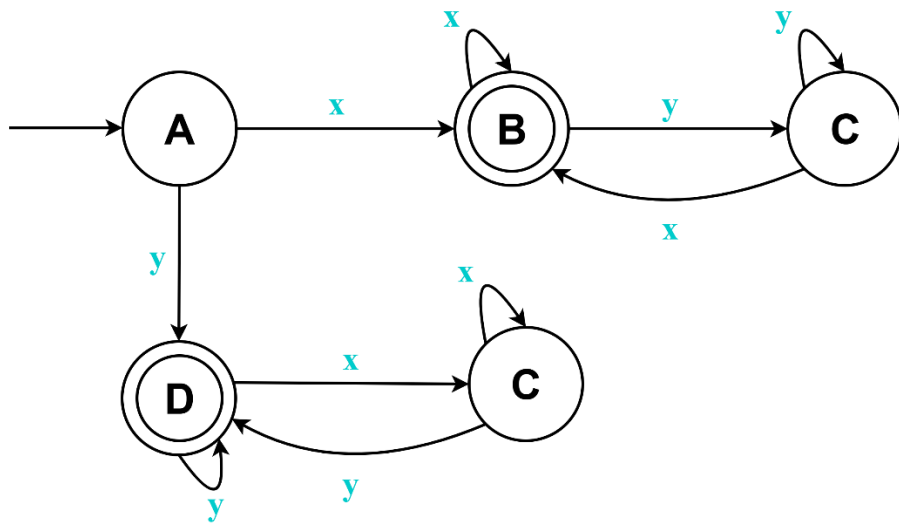
Accepted

(ii)

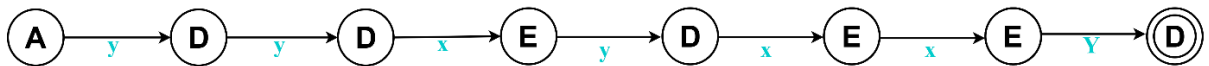
→ Is Given,

$\Sigma = \{x, y\}$ ,

$L(M1) = \{yxxxy, yxxxyxy, xyxyxx, \dots\}$



Eg: yyxyxyxy



Accepted